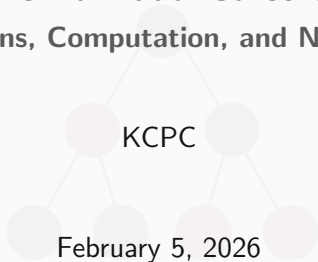


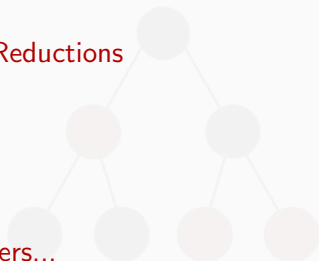
The Lambda Calculus

Functions, Computation, and Numbers



Overview

- 1 What is the Lambda Calculus?
- 2 Syntax of the Lambda Calculus
- 3 Alpha and Beta Reductions
- 4 Currying
- 5 Some Purity
- 6 We're Programmers...
- 7 Some More Complexity
- 8 Conclusion



Attendance



Vote for us!

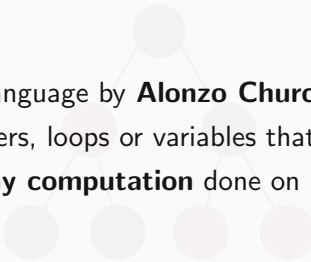


What is the Lambda Calculus?



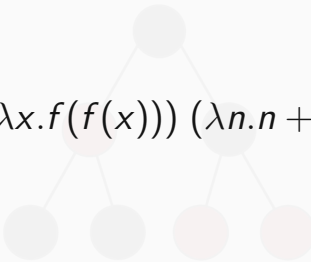
What is the Lambda Calculus?

Overview

- 
- Mathematical language by **Alonzo Church** in the 1930s
 - It has **no** numbers, loops or variables that store values
 - Can describe **any computation** done on a computer

What is the Lambda Calculus?

Example


$$(\lambda f. \lambda x. f(f(x))) (\lambda n. n + 1) 0$$

Evaluates to: 2

Goal of today: Understand how this works, and creating our own mathematics!

Syntax of the Lambda Calculus



What Is a Function?

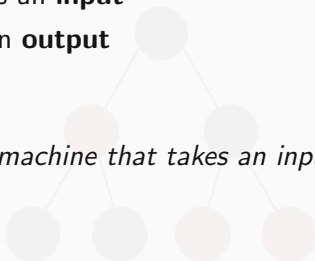
- A function takes an **input**
- And produces an **output**

Mental model:

A function is a machine that takes an input and produces an output

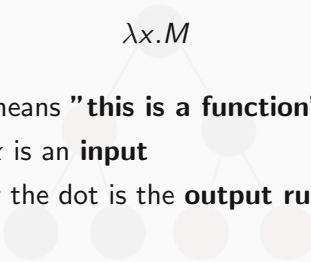
Important restriction:

- In the lambda calculus, a function takes **one input only**



Building a Function

In the lambda calculus, we build functions using **abstraction**.



$\lambda x.M$

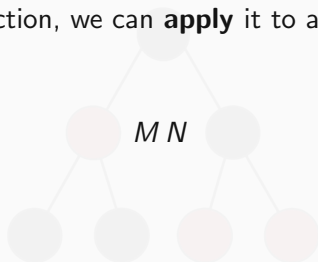
- The symbol λ means "**this is a function**"
- It tells us that x is an **input**
- Everything after the dot is the **output rule**

Why do we need λ ?

- To clearly say when a variable is an **input**
- To avoid confusion between inputs and ordinary names
- To make functions precise and unambiguous

Applying a Function

Once we have a function, we can **apply** it to an input.



→ M is a function

→ N is its input

→ Application means "run the machine"

The Three Building Blocks

In the lambda calculus, **every valid expression** is called a **term**. A term is

1 A variable

A name represents an input.

2 A function (abstraction)

A function that takes input x and returns M .

3 A function application

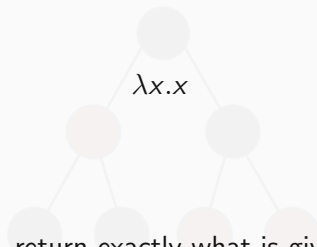
Apply the function M to the argument N .

Example: Identity Function

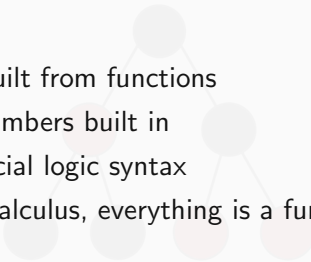
→ Input: x

→ Output: x

→ **Interpretation:** return exactly what is given



Functions, Functions Everywhere

- 
- Every term is built from functions
 - There are no numbers built in
 - There is no special logic syntax
 - In the lambda calculus, everything is a function

Alpha and Beta Reductions

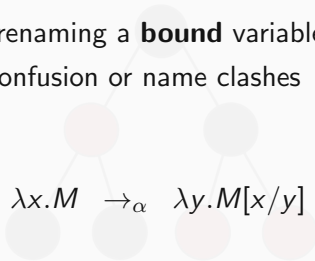


Alpha Conversion

Alpha conversion is:

- The process of renaming a **bound** variable
- Used to avoid confusion or name clashes

General form:



The diagram illustrates the general form of alpha conversion. It features a central equation $\lambda x.M \rightarrow_{\alpha} \lambda y.M[x/y]$. Above the equation, there are two light gray circles connected by a line. Below the equation, there are four circles: two light gray circles on the left and two light red circles on the right. Lines connect the top gray circle to the two bottom gray circles, and the bottom gray circle to the two bottom red circles. Another line connects the top gray circle to the two bottom red circles, forming a triangular structure that highlights the renaming of the bound variable x to y .

$$\lambda x.M \rightarrow_{\alpha} \lambda y.M[x/y]$$

Important rule:

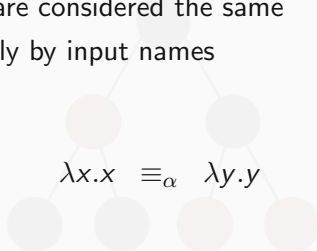
- Renaming must not change how the input is used

Alpha Equivalence

Alpha equivalence means:

- Two functions are considered the same
- If they differ only by input names

Example:



Key idea:

- Variable names are just labels
- What matters is how inputs are used

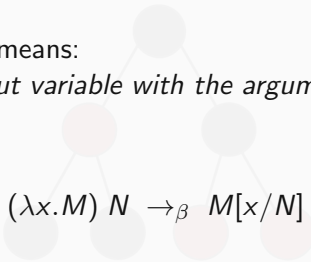
Beta Reduction

Beta reduction is where computation happens.

Applying a function means:

Replace the input variable with the argument

General rule:



A diagram illustrating the components of the general rule. It shows a tree structure with five nodes. The root node is a light gray circle. It has two children: a light red circle on the left and a light gray circle on the right. The left light red circle has two children: a light gray circle on the left and a light gray circle on the right. The right light gray circle has two children: a light red circle on the left and a light red circle on the right. This diagram visually represents the structure of the expression $(\lambda x.M) N$, where the root is the application, the left child is the lambda abstraction, and the right child is the argument. The lambda abstraction's children are the variable x and the body M , and the argument's children are the variable x and the body N .

$$(\lambda x.M) N \rightarrow_{\beta} M[x/N]$$

Example:

$$(\lambda x.x + 1) 3 \rightarrow 3 + 1 \rightarrow 4$$

Currying



Functions Take One Input

- In the lambda calculus, every function takes **exactly one input**
- There are no two-input or three-input functions

Problem:

- Many things in maths need more than one input
- For example: addition, multiplication

The Idea Behind Currying

Solution:

A function can return another function

- First input is given
- A new function is returned
- That new function waits for the next input

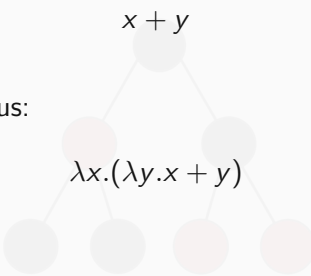
This idea is called currying.

Currying Explained with Addition

In ordinary maths:

$$x + y$$

In the lambda calculus:

$$\lambda x. (\lambda y. x + y)$$


- The outer function takes x
- It returns a new function
- The inner function takes y

Currying in Action

First apply the function to 1:

$$(\lambda x. \lambda y. x + y) 1 \rightarrow \lambda y. 1 + y$$


Then apply the result to 2:

$$(\lambda y. 1 + y) 2 \rightarrow 3$$


Conclusion:

- One input at a time
- But still behaves like a two-input function

Some Purity



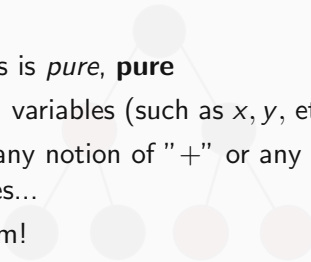
Small Issue

So far, we've used some *notation* without actually defining them...

→ $\lambda x.(\lambda y.x + y)$: what is a "+"?

→ $(\lambda y.1 + y) 2 \rightarrow 3$: what is a "2"? what is a "number"?

Purest of all Purest

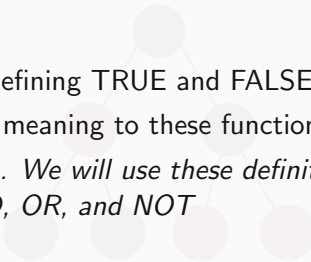
- 
- Lambda calculus is *pure*, **pure**
 - We only have λ , variables (such as x, y , etc.), and "(" and ")"
 - We don't have any notion of "+" or any of the numbers, or even truth values...
 - Let's create them!

We're Programmers...



We're Programmers...

...Bits!

- 
- Let's begin by defining TRUE and FALSE
 - Let's give some meaning to these functions
 - *Foreshadowing... We will use these definitions to create the operations AND, OR, and NOT*

We're Programmers...

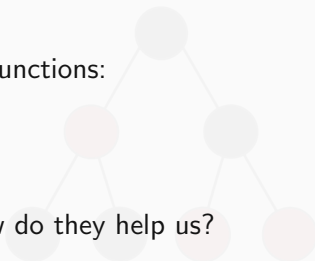
Some Hint?

Think of these two functions:

→ $\lambda x. \lambda y. x$

→ $\lambda x. \lambda y. y$

What are they? How do they help us?



We're Programmers...

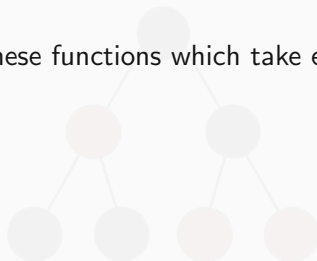
We Orn't And

Let's try to define these functions which take either a single or two parameters:

→ AND : (x, y)

→ OR : (x, y)

→ NOT : (x)



Some More Complexity

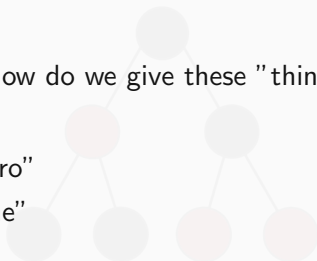


Unnaturally Natural Numbers

0, 1, 2, 3, 4, 5, ... How do we give these "things" meaning via lambda calculus?

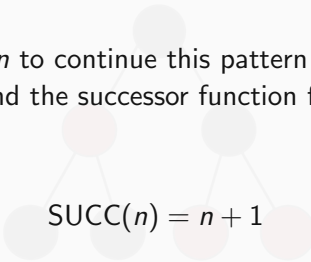
→ Let's define "zero"

→ Let's define "one"



Oh, I Know This!

We can use *induction* to continue this pattern and to define all natural numbers. Find the successor function for *your* numbering system:


$$\text{SUCC}(n) = n + 1$$

Finally Some Mathematics

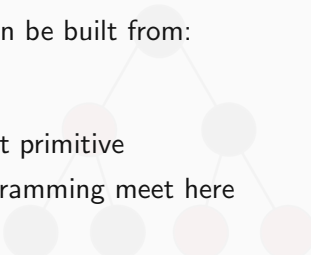
Using what we discovered and learned so far, let's finalise "our" mathematics:

- ADD : $(x, y) \rightarrow x + y$
- SUB : $(x, y) \rightarrow x - y$
- MUL : $(x, y) \rightarrow x \times y$
- POW : $(x, y) \rightarrow x^y$
- DIV : $(x, y) \rightarrow x/y$ (if you can do this without looking it up, I'll give you a candy)

Conclusion



What You've Learned

- 
- Computation can be built from:
 - Functions
 - Substitution
 - Numbers are not primitive
 - Maths and programming meet here

This is also the foundation of functional programming